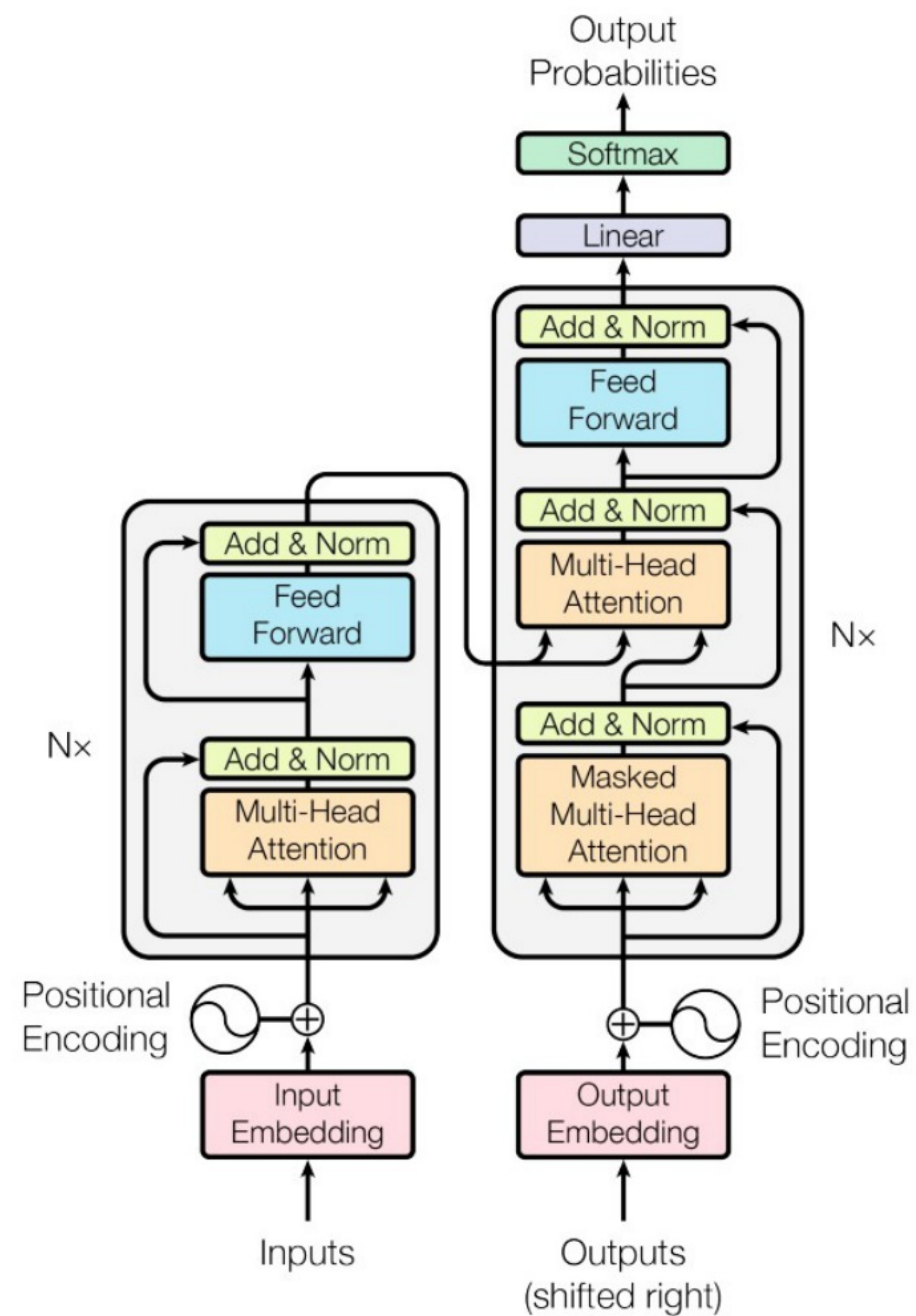


Трансформер Attention Is All You Need

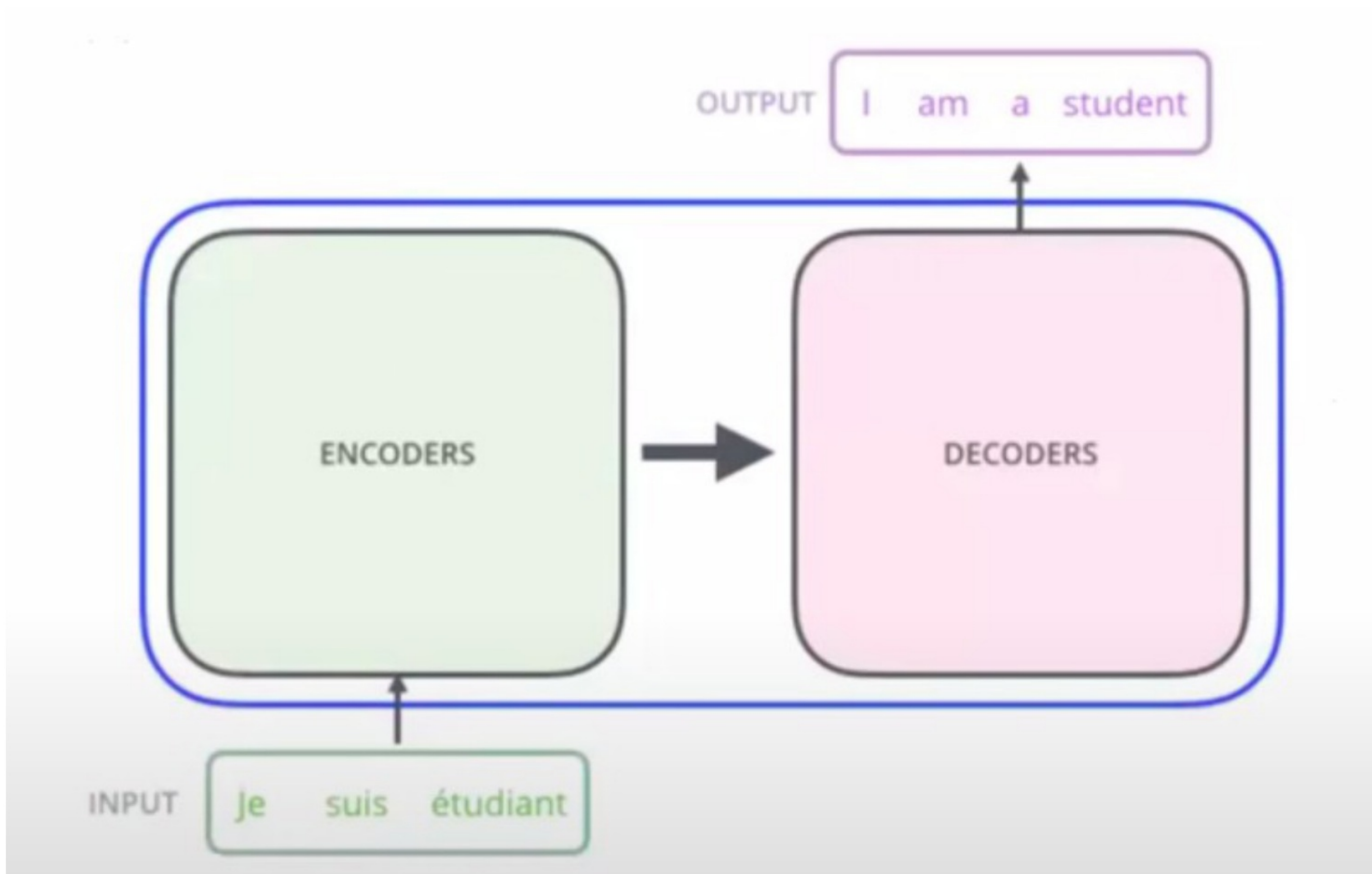
Attention is All You Need! (2017)

- RNN это очень долго! Всегда, чтобы найти следующий токен, надо знать предыдущий
- Backward pass идёт ещё и через время :(
- Transformer — нейросетевая архитектура для задач seq2seq, основанная исключительно на полносвязных слоях
- Превзошла существовавшие seq2seq архитектуры как по качеству, так и по скорости работы
- Основной элемент — multi-head self-attention

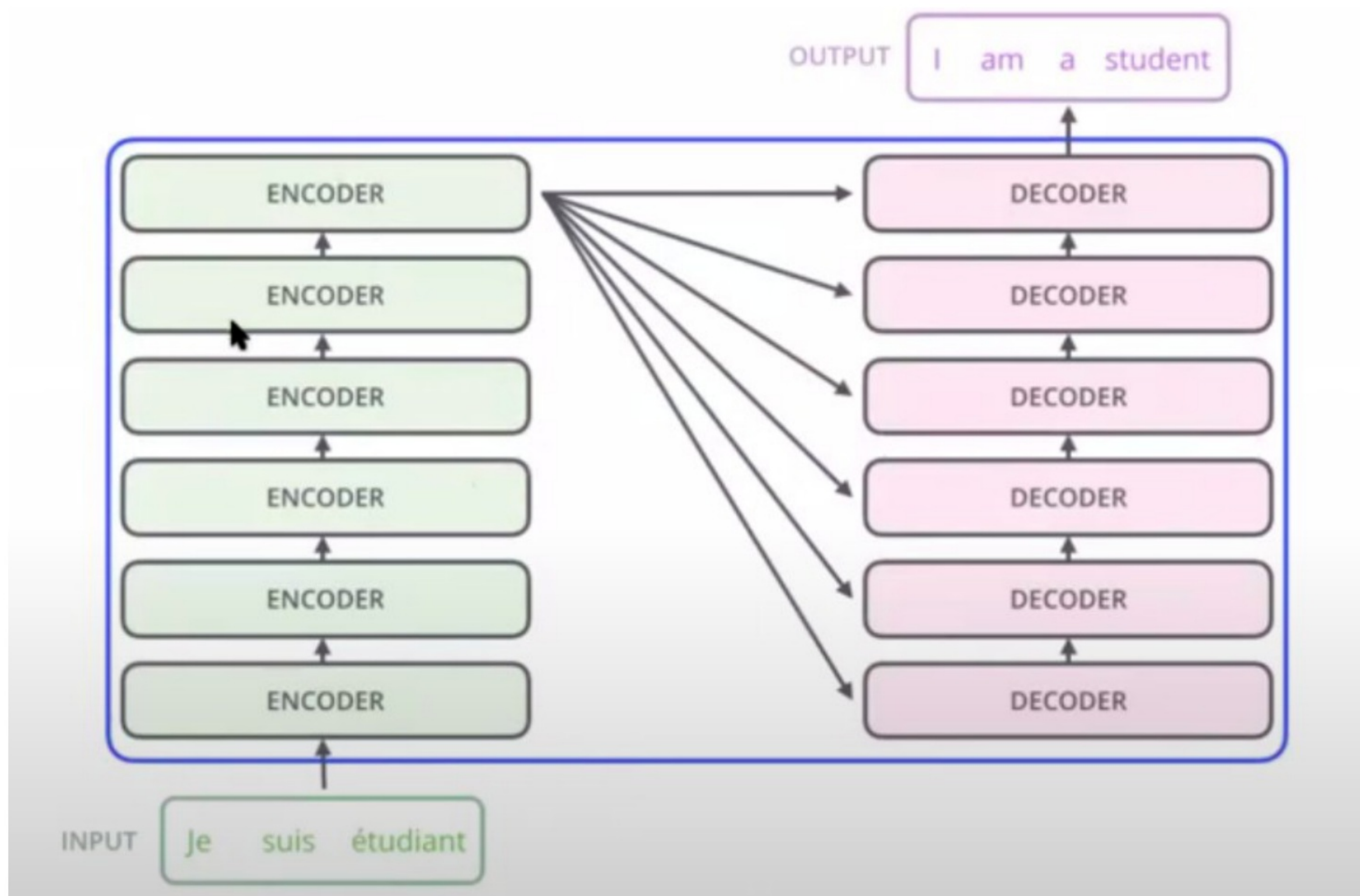
Архитектура трансформера



Трансформер (верхнеуровнево)



Трансформер (верхнеуровнево)

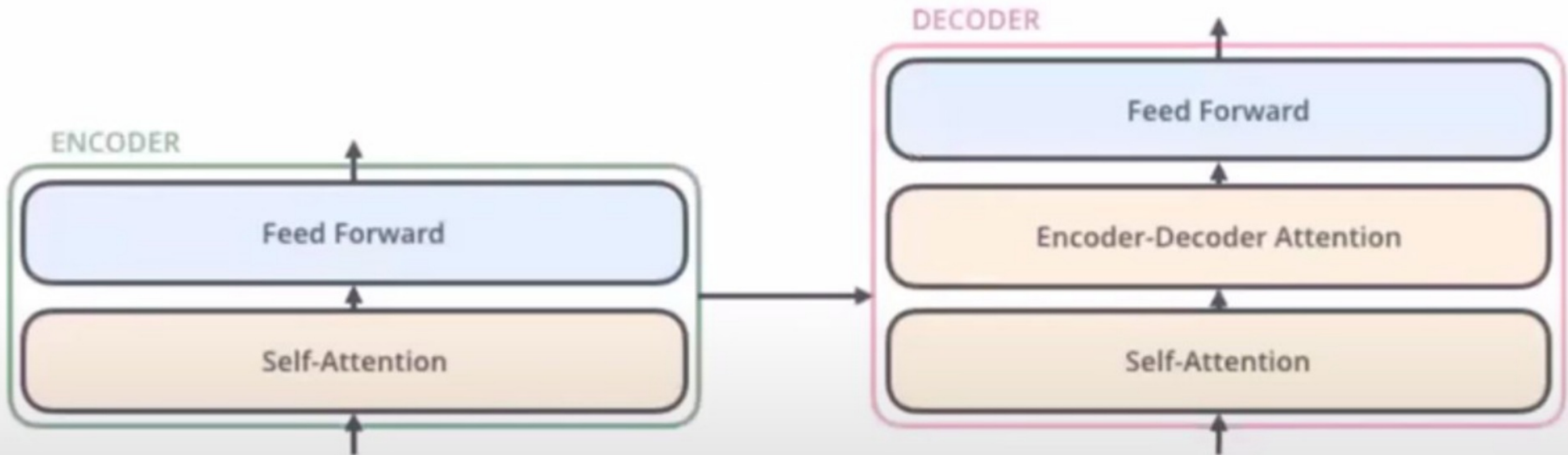


Трансформер

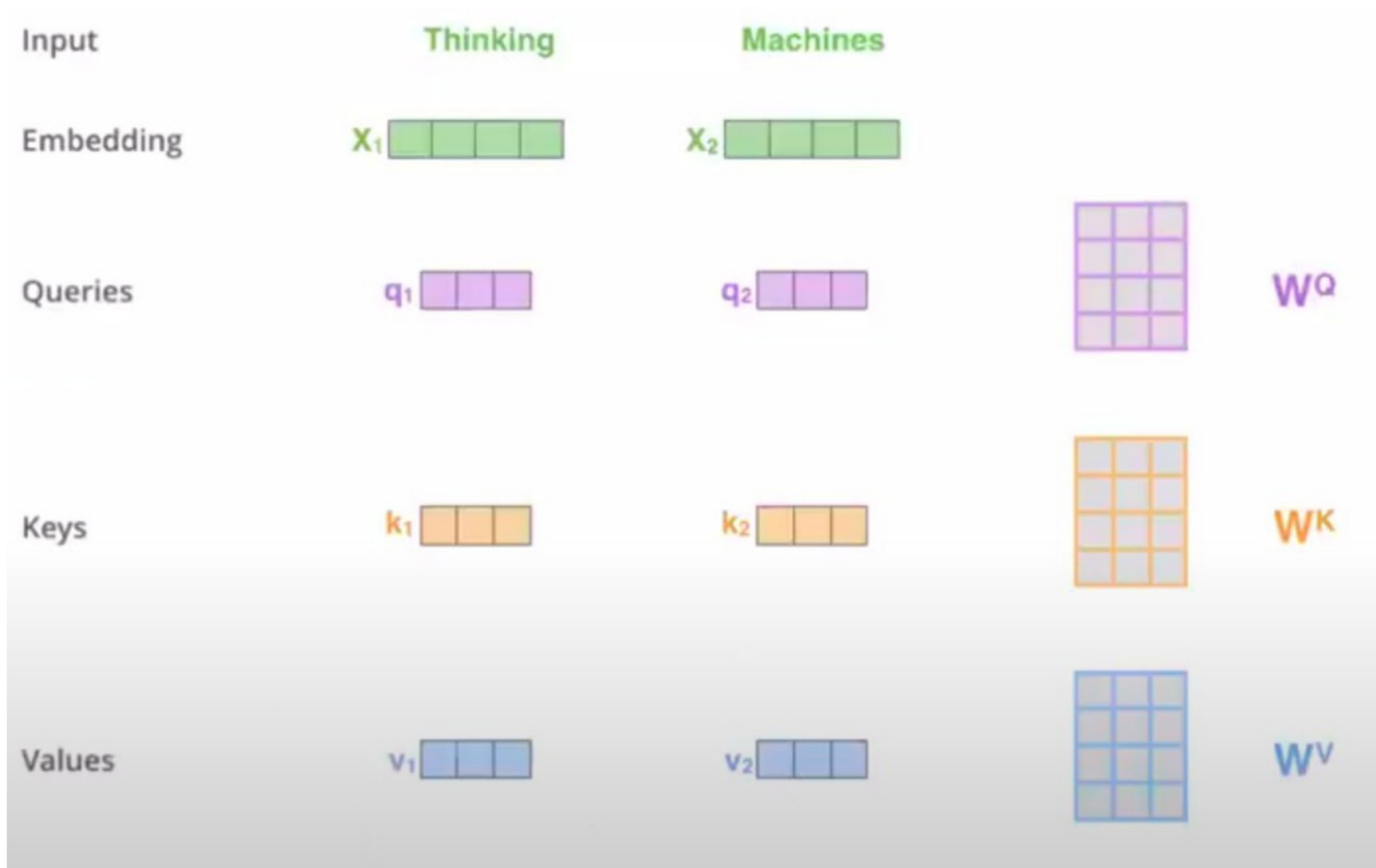
В энкодере происходят две вещи:

- Входные эмбединги прогоняются через self-attention (несколько раз)
- Затем пропускаем тензоры через полносвязный слой

В декодере помимо механизма self-attention есть механизм маскирования и attention из энкодера



Self-Attention



Self-Attention

- Для каждого входного слова считаются три вектора: Query, Key и Value
- Матрицы W^Q , W^K , W^V обучаются вместе с моделью
- Value - то, что мы знаем об этом слове
- Query, Key помогают искать связи между словами, мы ходим по всем словам и пытаемся понять насколько они связаны между собой
- Query - мое текущее слово, Key - мое слово с которым я сравниваю себя

Self-Attention

Цель этого слоя — сложить Value с некоторыми весами

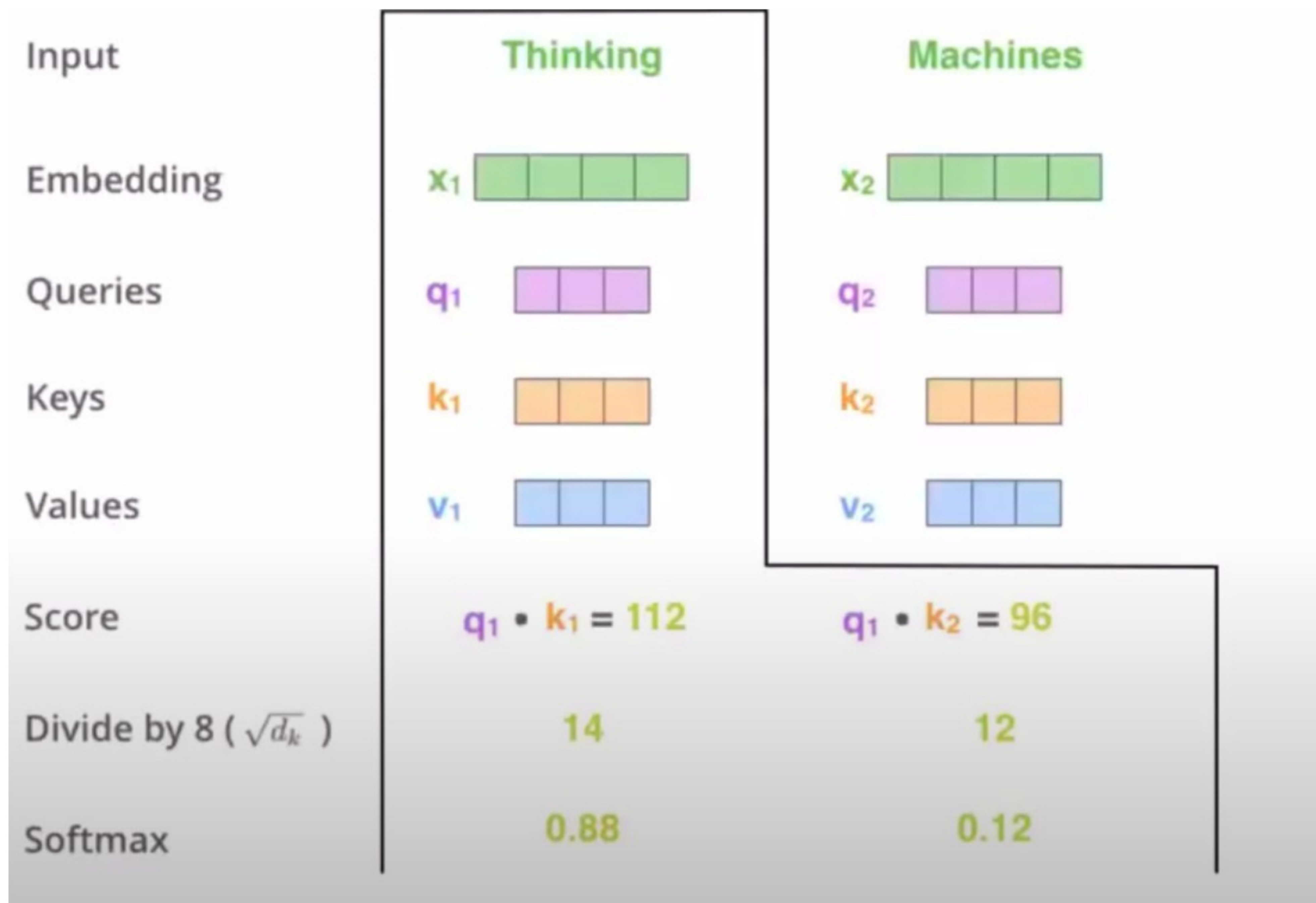
$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

=

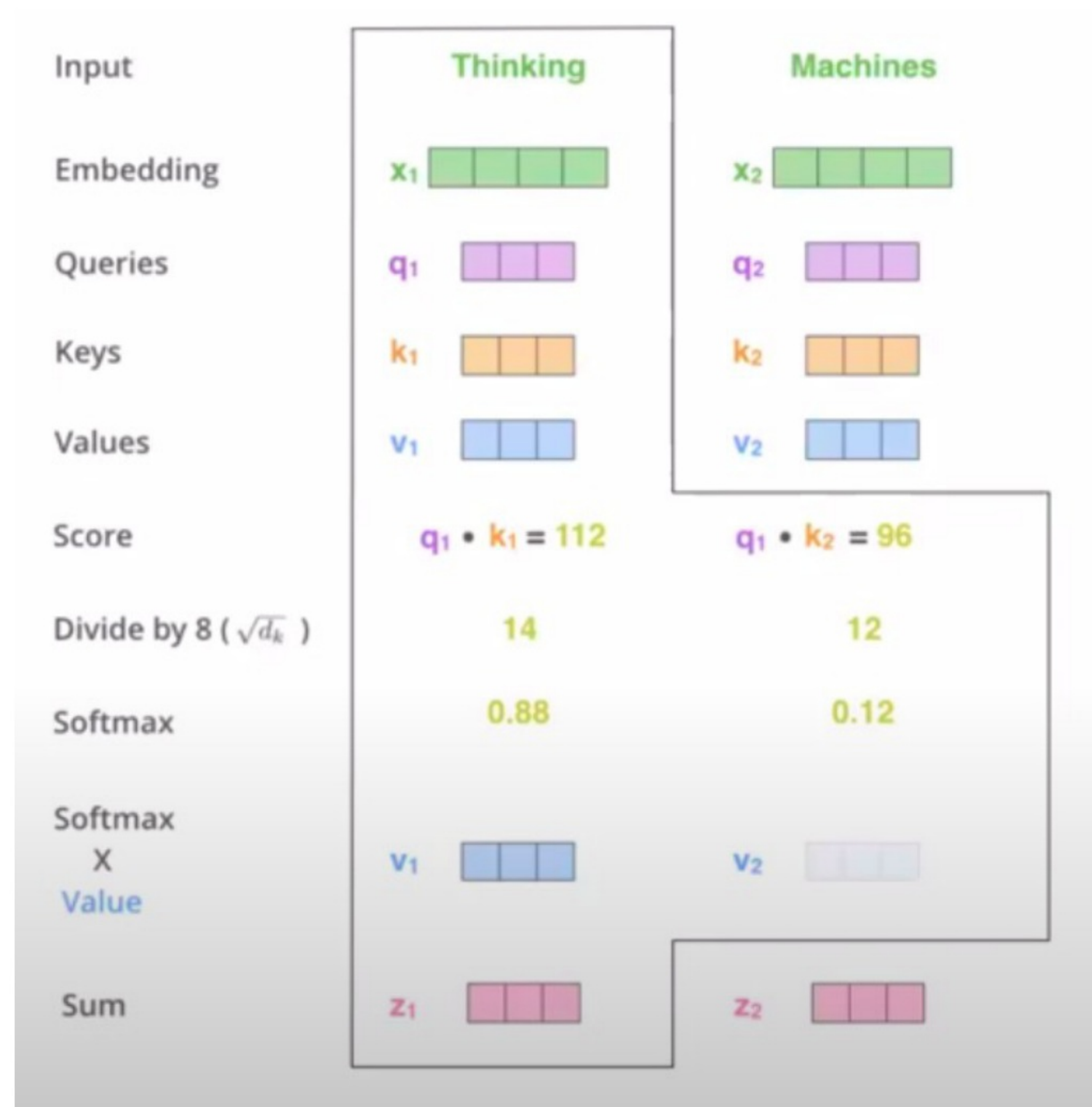
Z

$\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

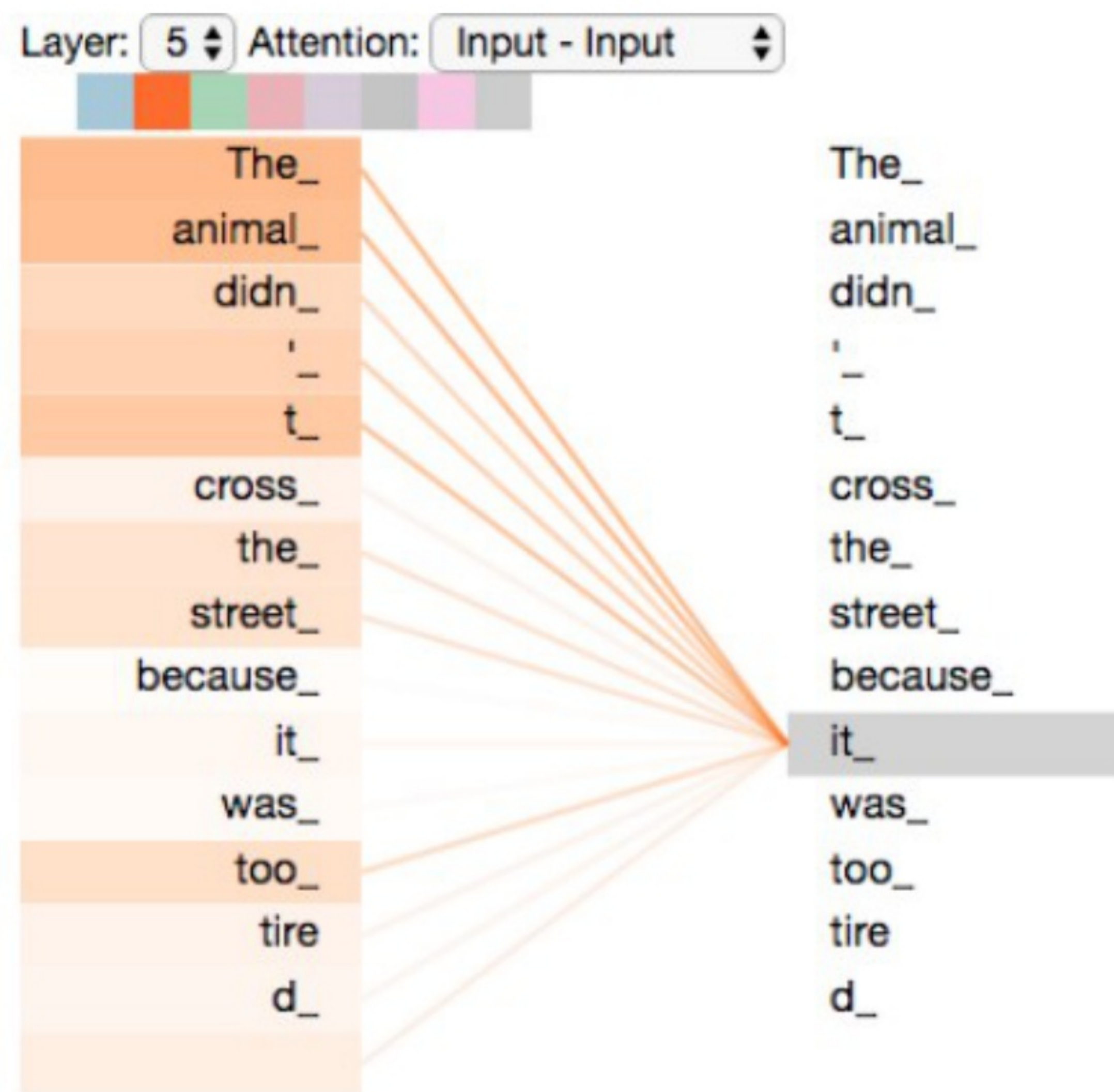
Self-Attention



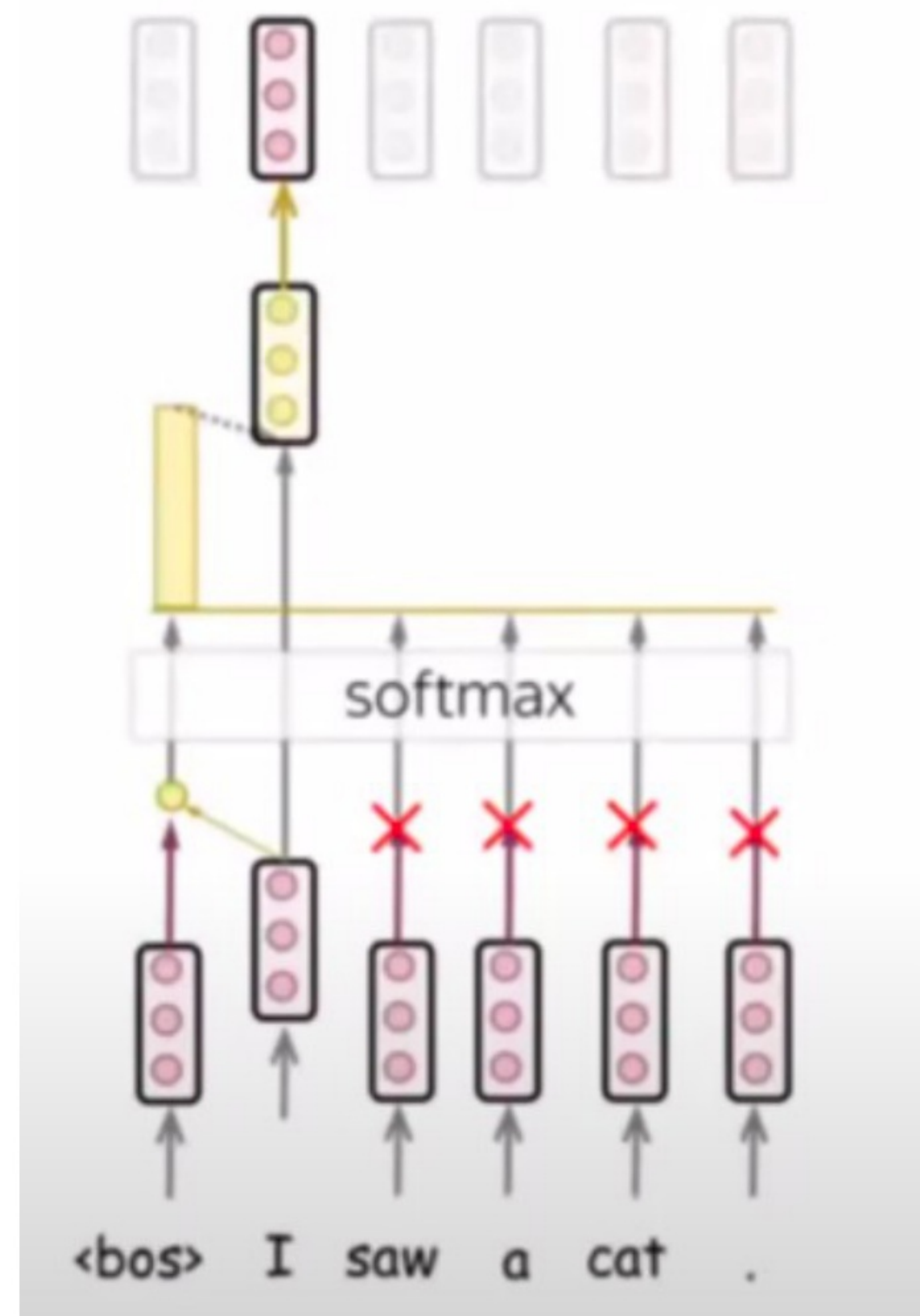
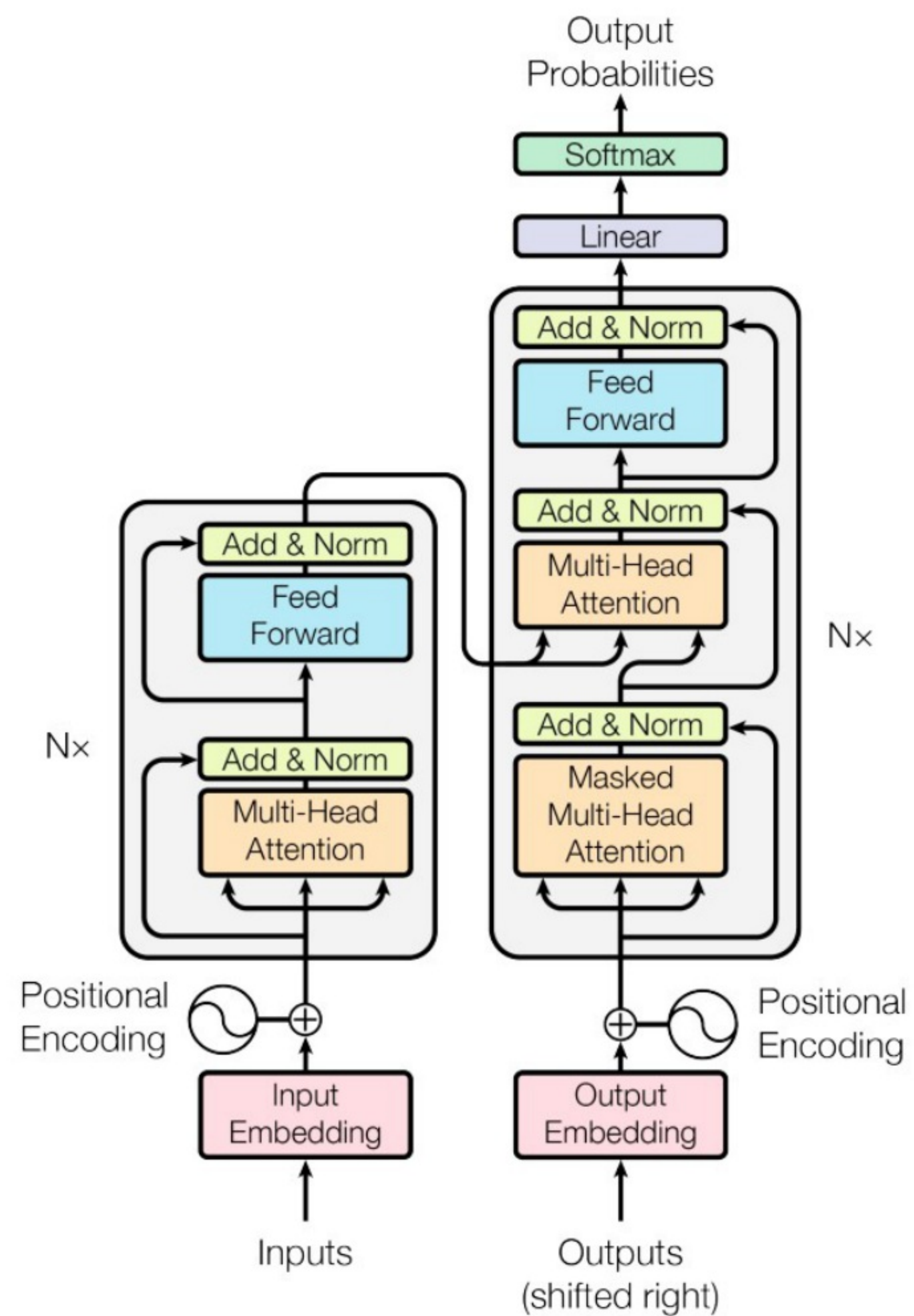
Self-Attention



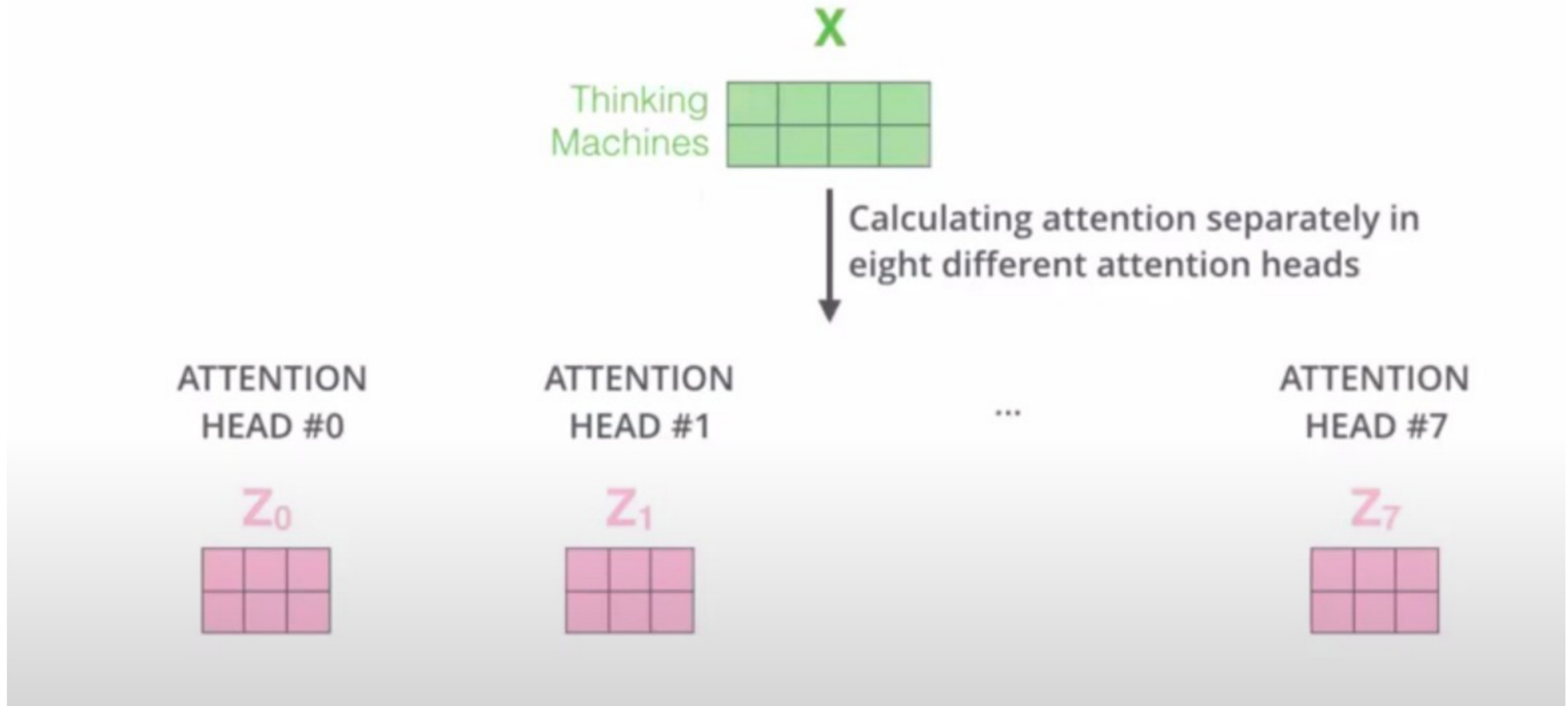
Визуализация Self-Attention



Маскирование



Multi-Head Attention



Слой целиком

1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads. We multiply X or R with weight matrices



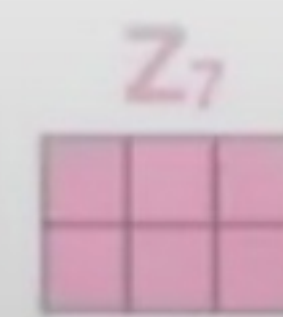
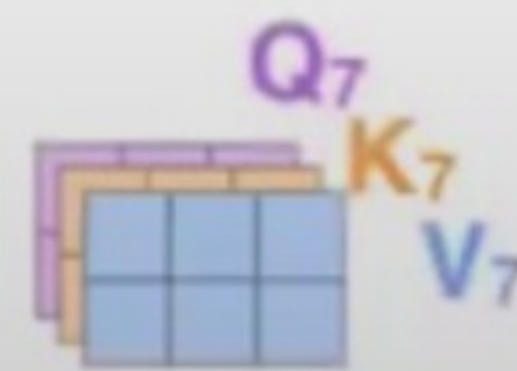
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

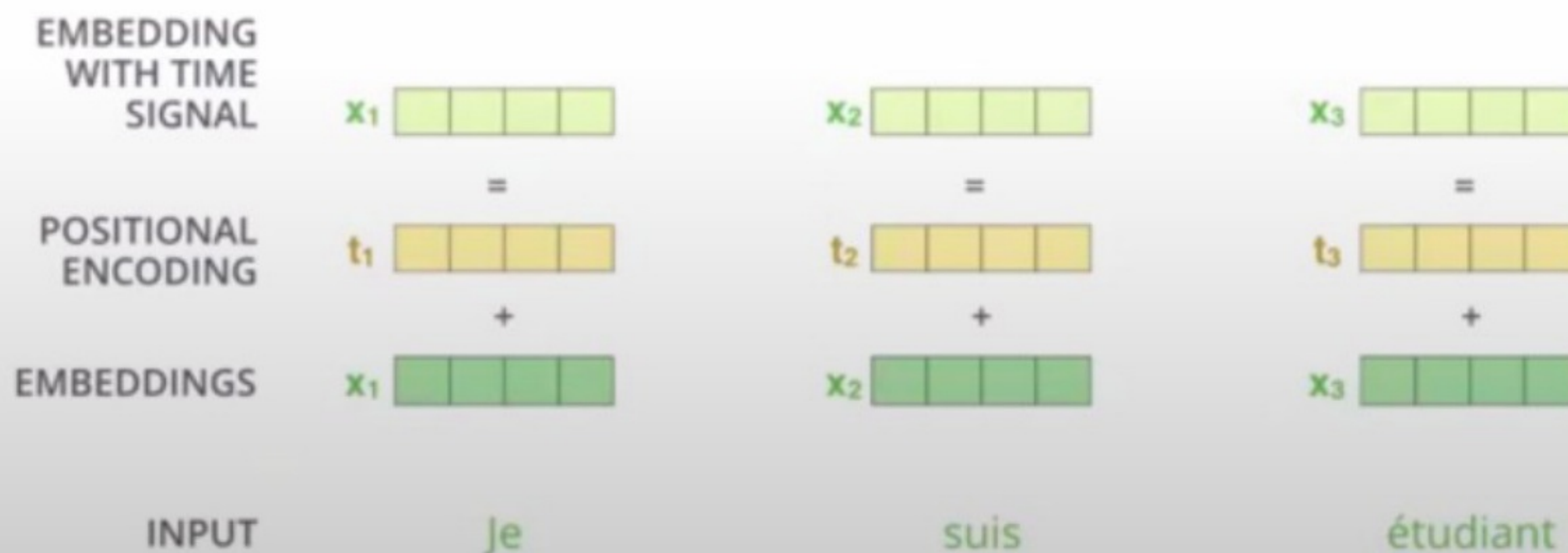
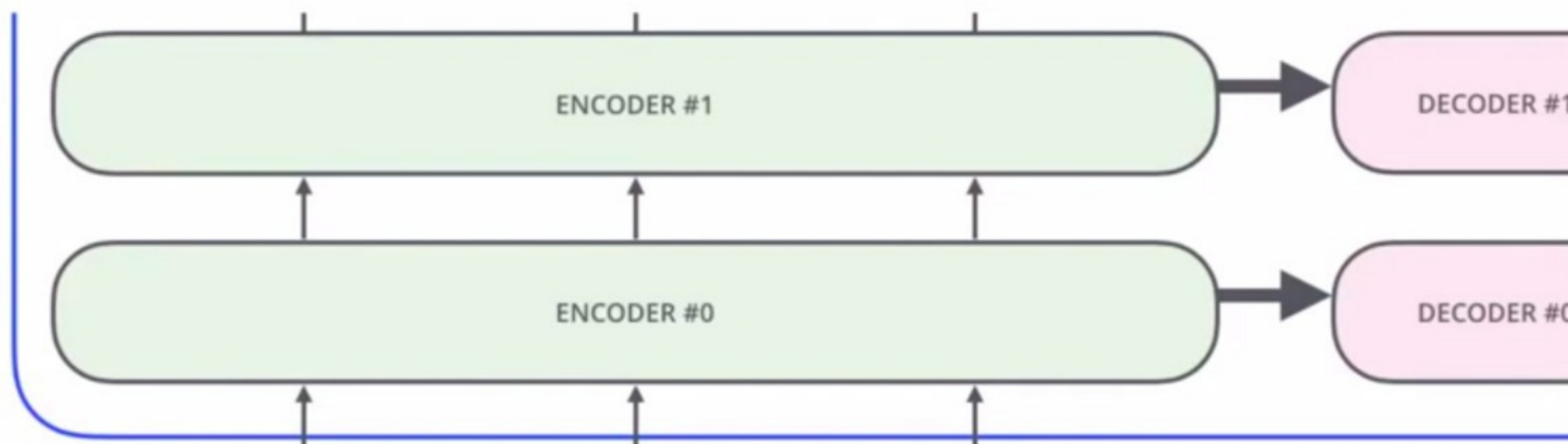


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



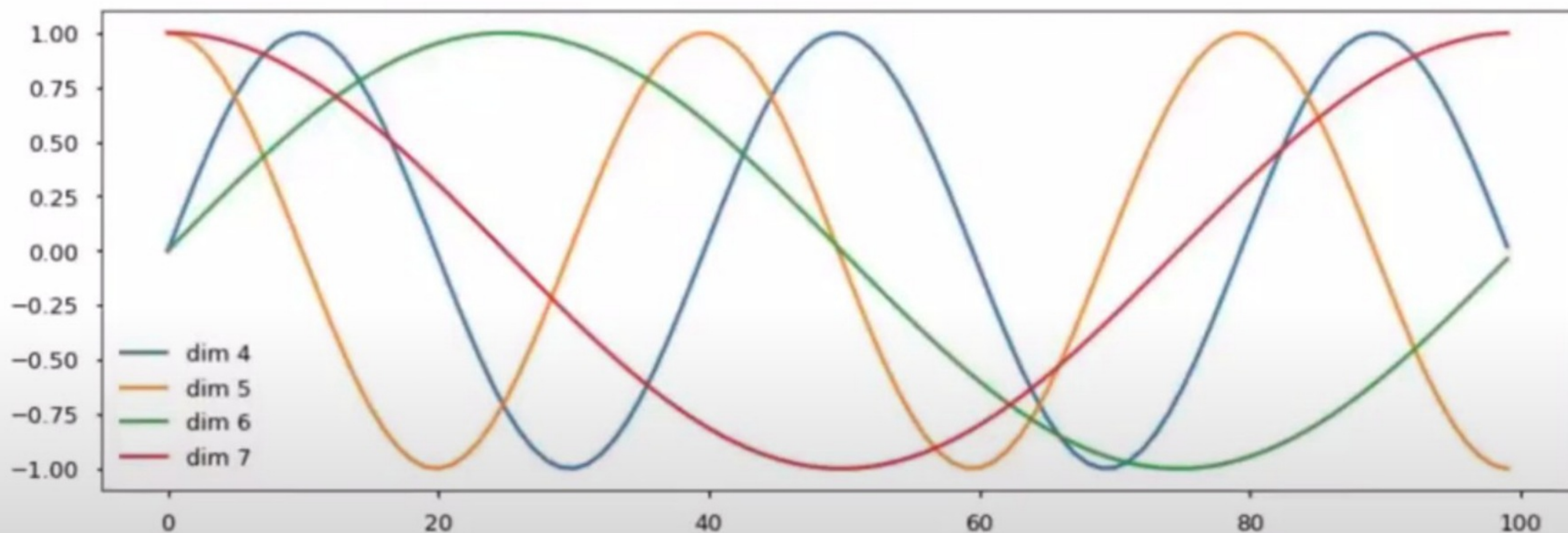
Positional encoding

Для учёта позиции можно приплюсовать дополнительный вектор



Positional encoding

Можно закодировать позицию с помощью синусоиды или какого-нибудь ONE-вектора



Positional encoding

Можно обучить под позицию отдельный эмбединг

Fixed encodings:

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

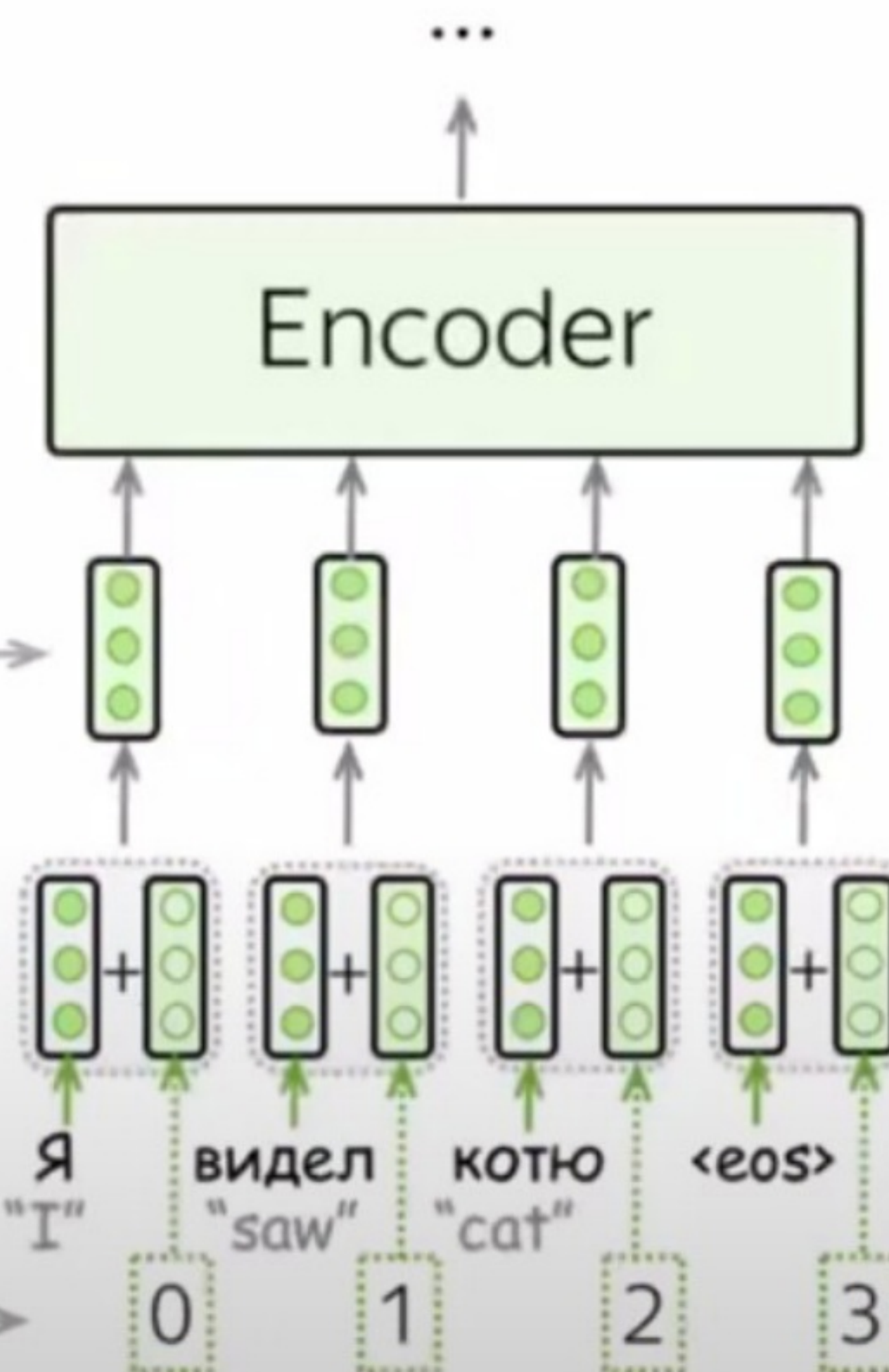
pos – position, i – dimension

“token x on position k” →

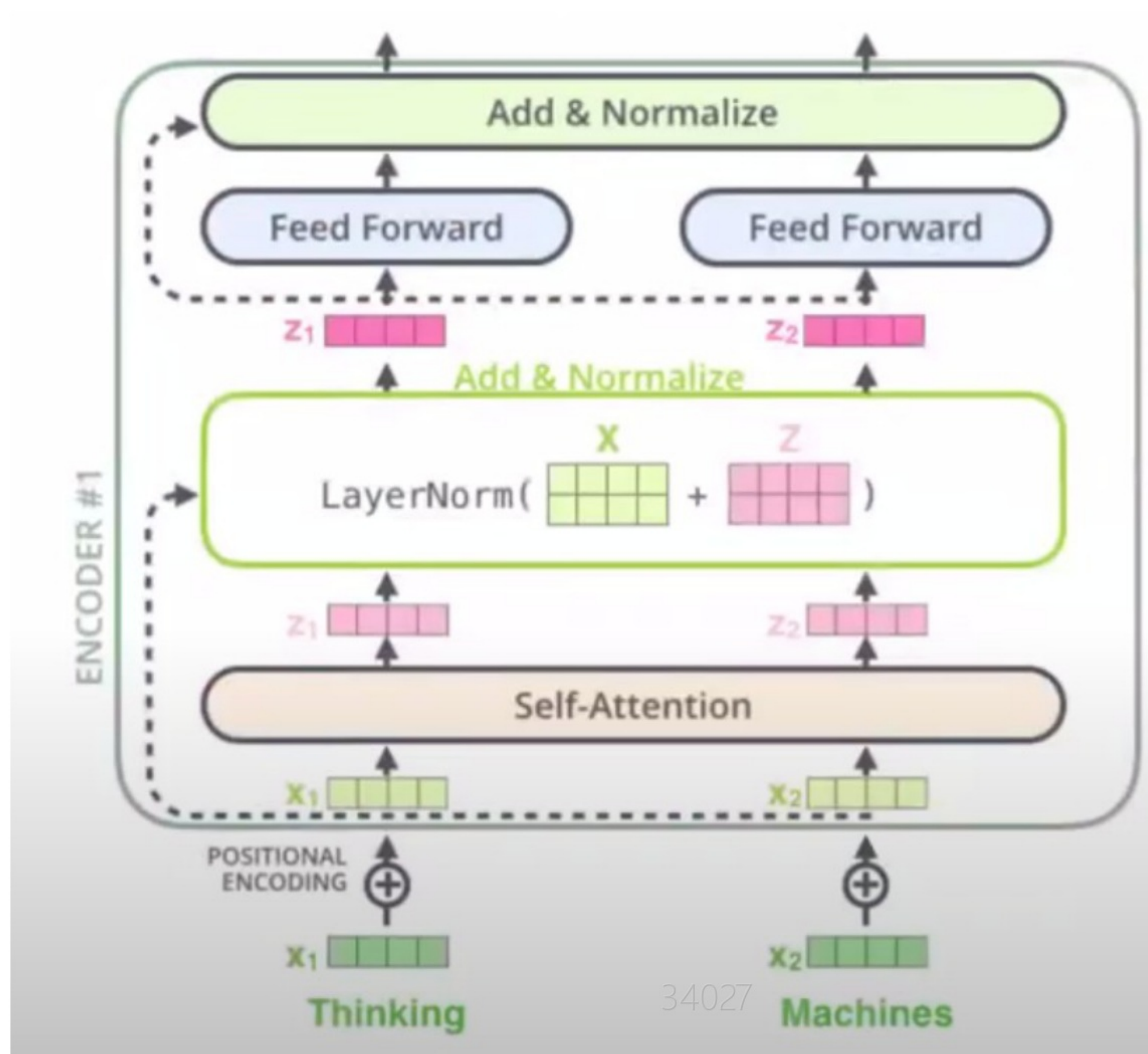
Input is sum of two embeddings: for token and position →

tokens →

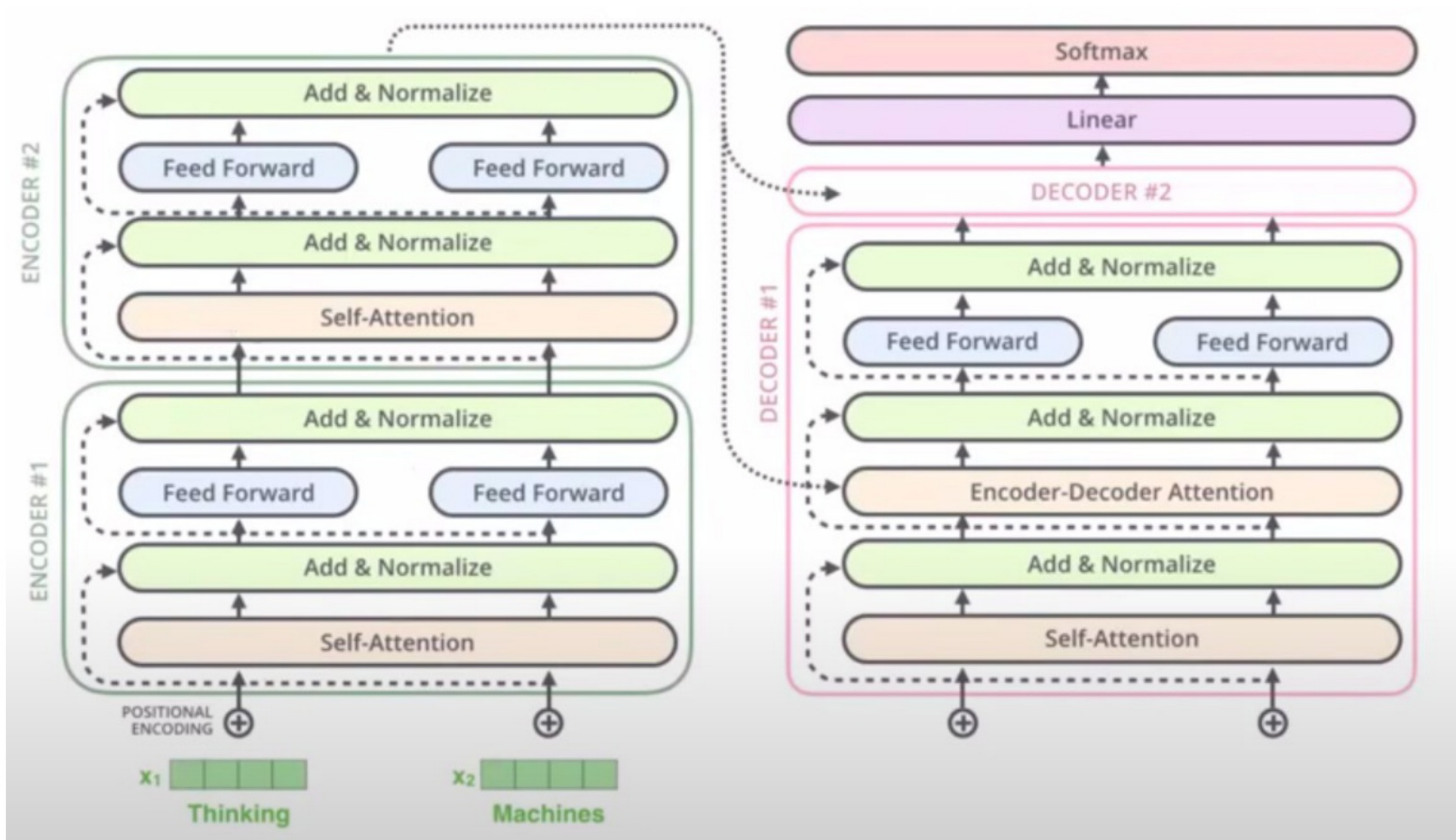
positions →



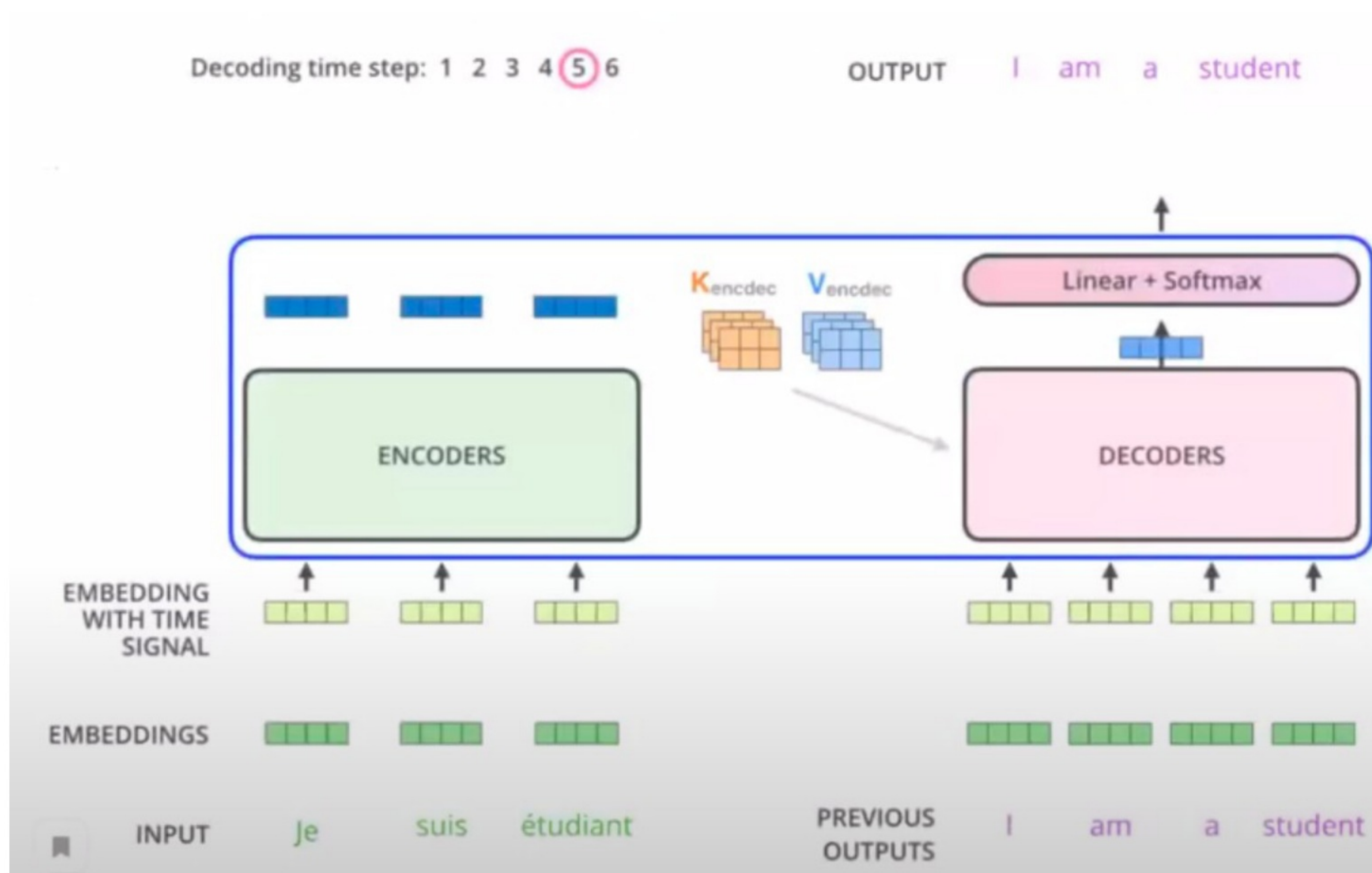
Что происходит в энкодере



Две модели рядом



Что происходит в декодере



Архитектура трансформера

